SQL Practice Assignment 5: Joins, Aliases & Unions

Topic: INNER JOIN, LEFT JOIN, Table Aliases, and Set Operations (UNION)

Role: Junior Data Analyst at "TechTrendz Retail"

Part 1: Expanding the Database (Setup)

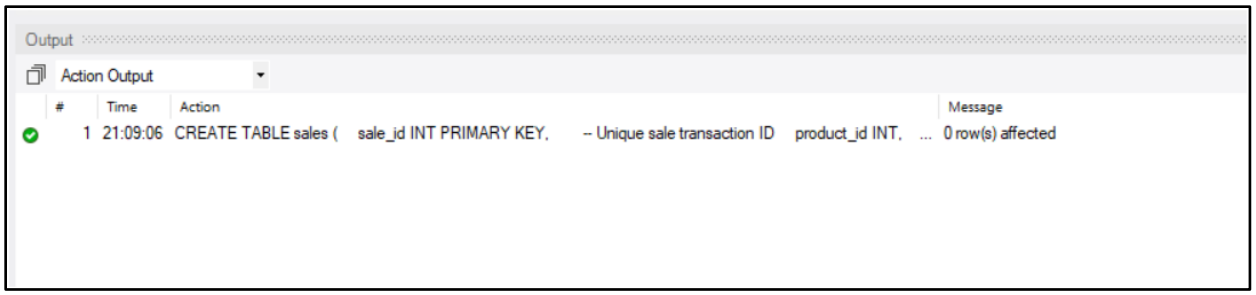
Context: To track revenue, we need a second table. The IT team has provided a sales log. We also have an old backup file of archived_products from last year.

Step 1: Create the Sales Table

Write a SQL query to create a table named sales with:

- sale_id (Integer, Primary Key)
- product_id (Integer) - *This links to your products table!*
- quantity_sold (Integer)
- sale_date (Date)

```
CREATE TABLE sales (
    sale_id INT PRIMARY KEY,        -- Unique sale transaction ID
    product_id INT,                 -- Links to product_id in products table
    quantity_sold INT,              -- Number of units sold
    sale_date DATE                  -- Date of sale
);
```



Step 2: Insert Sales Data

Insert the following transaction records:

| sale_id | product_id | quantity_sold | sale_date |

| :--- | :--- | :--- | :--- |

| 1 | 1 | 2 | 2023-10-01 |

| 2 | 3 | 1 | 2023-10-01 |

| 3 | 1 | 1 | 2023-10-02 |

| 4 | 12 | 5 | 2023-10-03 | | 5 | 10
| 1 | 2023-10-03 |
| 6 | 4 | 3 | 2023-10-04 |

| 7 | 5 | 2 | 2023-10-05 |

```
INSERT INTO sales VALUES
(1,1,2,'2023-10-01'),
(2,3,1,'2023-10-01'),
(3,1,1,'2023-10-02'),
(4,12,5,'2023-10-03'),
(5,10,1,'2023-10-03'),
(6,4,3,'2023-10-04'),
(7,5,2,'2023-10-05');
```

Output

Action Output

#	Time	Action	Message
1	21:09:06	CREATE TABLE sales (sale_id INT PRIMARY KEY, -- Unique sale transaction ID product_id INT, ...	0 row(s) affected
2	21:15:35	INSERT INTO sales VALUES (1,1,2,'2023-10-01'), (2,3,1,'2023-10-01'), (3,1,1,'2023-10-02'), (4,12,5,'2023-10-...	7 row(s) affected Records: 7 Duplicates: 0 Warnings: 0

Step 3: Create & Populate Archive Table (For Unions)

Create a table named archived_products and insert these rows.

```
-- Create archived_products table for old product records
CREATE TABLE archived_products (
   product_id INT,
   product_name VARCHAR(100),
   price DECIMAL(10,2)
);
```

Output

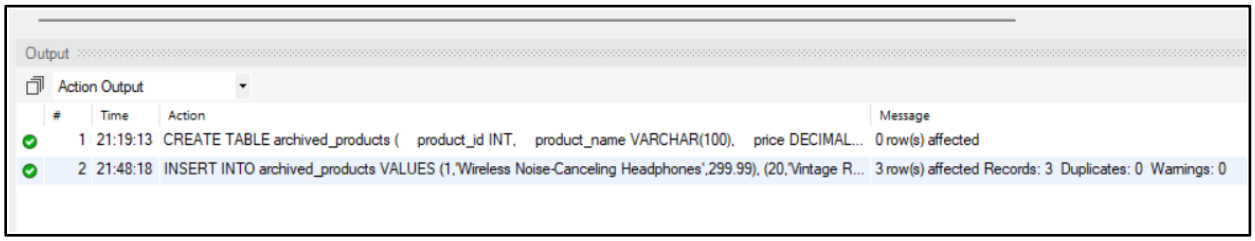
Action Output

#	Time	Action	Message
1	21:19:13	CREATE TABLE archived_products (product_id INT, product_name VARCHAR(100), price DECIMAL...	0 row(s) affected

Note: Product ID 1 exists in both the current and archive tables. This is intentional.

product_id	product_name	price
1	Wireless Noise-Canceling Headphones	299.99
20	Vintage Record Player	150.00
21	CRT Monitor 19"	45.00

```
-- Insert archived product records
INSERT INTO archived_products VALUES
(1,'Wireless Noise-Canceling Headphones',299.99),
(20,'Vintage Record Player',150.00),
(21,'CRT Monitor 19"',45.00);
```



Part 2: Analysis Questions

A. The Power of Joins (INNER JOIN)

Context: We need to see human-readable names alongside our sales numbers, not just ID numbers.

1. **Sales Detail Report:**
   - Select sale_date, product_name, and quantity_sold.
   - ○ Join the sales table with the products table. ○ **Requirement:** Use an INNER JOIN. (INNER JOIN returns **matching records from both tables**.)

```
-- Show sale date, product name, and quantity sold
-- Join sales table with products table
SELECT s.sale_date,
       p.product_name,
       s.quantity_sold
```

```
FROM sales s
INNER JOIN products p
ON s.product_id = p.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

sale_date	product_name	quantity_sold
2023-10-01	Wireless Noise-Canceling Headphones	2
2023-10-01	Gaming Laptop Pro	1
2023-10-02	Wireless Noise-Canceling Headphones	1
2023-10-03	Cotton T-Shirt	5
2023-10-03	Coffee Maker	1
2023-10-04	Stainless Steel Water Bottle	3
2023-10-05	Bluetooth Speaker Mini	2

2. **High-Value Transactions:**
   - Select product_name and the **Total Revenue** for that specific sale (quantity_sold * price).
   - Filter to show only transactions worth **more than $500**.

```
-- Find sales where total revenue is greater than $500

-- revenue = quantity_sold * price


SELECT p.product_name,

 s.quantity_sold * p.price AS total_revenue

FROM sales s

INNER JOIN products p

ON s.product_id = p.product_id

WHERE s.quantity_sold * p.price > 500;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	total_revenue
Wireless Noise-Canceling Headphones	599.98
Gaming Laptop Pro	1200.00

B. Finding Missing Data (LEFT JOIN)

Context: Inner Joins only show matches. Left Joins show everything from the left table, even if there is no match.

3. **The "Unsold Items" Report:**
   - Management wants to know which products have **never** been sold.
   - Select product_name and sale_id.
   - Perform a LEFT JOIN starting with products (Left) to sales (Right).
   - Filter the results to only show rows where sale_id **IS NULL**.
   - *Interview Hint: This is the standard way to find "records with no matches" in SQL.*

```
-- Find products that were never sold

-- LEFT JOIN shows all products even if no sales exist

-- Shows products with no matching sales records.

SELECT p.product_name, s.sale_id

FROM products p

LEFT JOIN sales s

ON p.product_id = s.product_id

WHERE s.sale_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	sale_id
Ergonomic Office Chair	NULL
Running Shoes - Size 10	NULL
Leather Wallet	NULL
4K Smart TV 55 inch	NULL
Yoga Mat	NULL
Smart Watch Series 5	NULL
Gaming Mouse	NULL
Desk Lamp	NULL
Ceramic Mug Set	NULL
Smart Glasses Prototype	NULL
Mystery Grab Bag	NULL

C. Professional Aliases

Context: Typing products.product_name is tedious. Use Aliases (p, s) to make code cleaner.

4. **Sales Summary with Aliases:**
   - Rewrite your query from Question 1.
   - Alias products as p and sales as s.

- Use the format p.product_name and s.quantity_sold in your SELECT and ON clauses.

-- Use aliases p (products) and s (sales)

-- Aliases make SQL shorter and easier to read.

SELECT s.sale_date, p.product_name, s.quantity_sold

FROM products p

INNER JOIN sales s

ON p.product_id = s.product_id;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

sale_date	product_name	quantity_sold
2023-10-01	Wireless Noise-Canceling Headphones	2
2023-10-01	Gaming Laptop Pro	1
2023-10-02	Wireless Noise-Canceling Headphones	1
2023-10-03	Cotton T-Shirt	5
2023-10-03	Coffee Maker	1
2023-10-04	Stainless Steel Water Bottle	3
2023-10-05	Bluetooth Speaker Mini	2

D. Set Operations (UNION vs UNION ALL)

Context: Joins combine columns (horizontal). Unions combine rows (vertical).

5. **Master Product Catalog:**
   - Create a single list of ALL product names from both the active products table and the archived_products table.
   - Use UNION.
   - *Question:* Does the "Headphones" entry appear once or twice?

-- Combine product names from active and archived tables
-- UNION removes duplicate rows
SELECT product_name
FROM products
UNION
SELECT product_name
FROM archived_products;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name
Wireless Noise-Canceling Headphones
Ergonomic Office Chair
Gaming Laptop Pro
Stainless Steel Water Bottle
Bluetooth Speaker Mini
Running Shoes - Size 10
Leather Wallet
4K Smart TV 55 inch
Yoga Mat
Coffee Maker
Smart Watch Series 5
Cotton T-Shirt
Gaming Mouse
Desk Lamp
Ceramic Mug Set
Smart Glasses Prototype
Mystery Grab Bag

Ans: Headphones appears only once because UNION removes duplicates.

6. **Historical Audit:**
   - Run the same query as above, but use UNION ALL.
   - *Question:* How does the result differ?

```
-- Combine product names but keep duplicates

SELECT product_name

FROM products

UNION ALL

SELECT product_name

FROM archived_products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name
Wireless Noise-Canceling Headphones
Ergonomic Office Chair
Gaming Laptop Pro
Stainless Steel Water Bottle
Bluetooth Speaker Mini
Running Shoes - Size 10
Leather Wallet
4K Smart TV 55 inch
Yoga Mat
Coffee Maker
Smart Watch Series 5
Cotton T-Shirt
Gaming Mouse
Desk Lamp
Ceramic Mug Set
Smart Glasses Prototype
Mystery Grab Bag
Wireless Noise-Canceling Headphones
Vintage Record Player
CRT Monitor 19"

Ans: Headphones appears twice because UNION ALL keeps duplicates.

E. Complex Challenge

7. **Revenue per Category:**
   - Calculate the total revenue (SUM(quantity_sold * price)) generated by each **Category**.
   - You will need to Join products and sales, then Group By category.
   - Order by revenue Descending.

```
-- Calculate total revenue generated by each category
-- revenue = quantity_sold * price
-- Join products with sales
-- Calculate revenue
-- Group by category
SELECT p.category,
    SUM(s.quantity_sold * p.price) AS total_revenue
FROM products p
INNER JOIN sales s
ON p.product_id = s.product_id
GROUP BY p.category
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_revenue
Electronics	2191.95
Appliances	75.50
Apparel	75.00
Accessories	75.00